

JSON and Byte File I/O

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Texts:** *Core Java, Volume II* (13th ed.), Chapter 2; the org.json library documentation
- **Prerequisite you already have:** basic classes and objects, and file input and output with text and CSV files from Week 3

1. The short version

This week covers two more ways to move data in and out of a Java program, both new this semester. The first is JSON, a structured text format read and written with the org.json library. The second is binary, storing data as raw bytes instead of characters. Both build on the file reading and writing you already know from Week 3; only the format on disk changes.

Day one covers org.json: building a JSONObject and JSONArray by hand, reading named values back out, and reading and writing whole JSON files. Day two covers byte files: writing primitive values directly as bytes with DataOutputStream, and reading them back in the exact same order with DataInputStream, since a binary file carries no field names or structure of its own the way JSON does.

Each day opens with a short demonstration, then moves into a small example we build together in class, with you directing how the problem gets solved. If we do not finish an example in class, it is fine to finish it on your own afterward for practice; the goal in class is to get you started, not necessarily to finish.

2. What We Will Cover

- **JSONObject and JSONArray.** Building both by hand with `put`, including a JSONArray nested inside a JSONObject.
- **Reading values back out.** The typed getter methods (`getString`, `getInt`, `getDouble`, `getBoolean`, `getJSONArray`), and what happens when the wrong one is used.
- **Reading and writing JSON files.** Reading a file's full text into a String to build a JSONObject, and writing a JSONObject back out, pretty-printed.
- **Converting between objects and JSON.** Moving data between a custom Java class and a JSONObject in both directions.
- **Byte streams.** `FileOutputStream` and `FileInputStream` for raw bytes, and why a binary file is not readable as plain text.
- **DataOutputStream and DataInputStream.** Writing primitive values (int, double, boolean, and String) as bytes, and reading them back in the exact order they were written.

- **Storing a list in a binary file.** Writing a count before a set of items, so the count tells a reader how many items to expect.

3. Objectives

By the end of this week, you should be able to:

1. Build a `JSONObject` and a nested `JSONArray` by hand, using `put`.
2. Read named values out of a `JSONObject` with the correct typed getter method for each field.
3. Read a JSON file's contents into a `String` and build a `JSONObject` or `JSONArray` from it.
4. Write a `JSONObject` or `JSONArray` to a file, pretty-printed for readability.
5. Convert a custom Java object to a `JSONObject` and back, field by field.
6. Explain, in your own words, the tradeoffs between a text-based format like JSON and a binary format: readability, size, and whether the file describes its own structure.
7. Write primitive values to a file with `DataOutputStream`, and read them back with `DataInputStream` in the same order.
8. Explain why binary reads must happen in the exact order the data was written, unlike JSON, where each value is found by name.
9. Write and read a variable-length list in a binary file, using a count written before the list's items.
10. Open and close file streams safely with `try-with-resources`, and handle `IOException` when a read or write fails.

4. How to Show Up Ready

- Review the Week 3 file I/O notes, especially how we read a CSV file and wrote a report back out. Both new formats this week reuse that same overall shape.
- Research how JSON files are organized. Look at a few real examples and get a feel for named fields, nested objects, and arrays, before we start building with `org.json` in class.
- Download the `org.json` library from its GitHub page, <https://github.com/stleary/JSON-java>, and practice setting it up in a project's `libs` folder, the same way we set up `OpenCSV` earlier this semester.
- Bring a laptop with your development environment ready, if you want to follow along during the class-directed examples.
- Come with questions about anything from Week 3's file I/O that still feels unclear. This week assumes that foundation is solid.